



中山大學
SUN YAT-SEN UNIVERSITY

多智能体集群控制（智能车部分）

实验二：联机多车实验

姓 名 _____ 党政扬

学 号 _____

学 院 _____ 智能工程学院

专 业 _____ 智能科学与技术

2026 年 7 月 6 日

摘要

本报告记录了实验二“联机多车实验”的完整实践过程。笔者首先完成了多车联机环境的搭建，将实验一验证过的单车控制程序（纯追踪 + Stanley 混合控制）接入联机房间，确认了状态接收与控制指令下发在多车场景下依然稳定。在此基础上，笔者拆解了指导书提供的示例避障函数，发现其以“车辆自身朝向”为参考系、以固定 $\pm 45^\circ$ 扇形做二值判定（发现障碍即刻置零）的做法存在三处缺陷：弯道处因参考系偏转而漏检本车道前车、二值急停造成明显的点头式顿挫、以及仅凭相对方位角判断而不含车道信息，导致面对相邻车道的对向来车时会把“其实不会相撞”的车辆误判为前方障碍。针对前两处缺陷，笔者将检测参考系由车辆朝向改为路径切线方向，并把二值急停改造为基于距离的分级线性减速；针对第三处缺陷，笔者在场景验证中专门复现了它的后果——直道双车道对向会车死锁：两车分处相邻车道、方向相反、本可安全交会，却因各自的 $\pm 45^\circ$ 扇形同时扫到对方而同时停车。笔者进一步说明改进方案中的横向车道阈值恰好能过滤掉这类邻道来车，从机理上缓解该死锁，并客观指出纯正面同车道冲突与对称对峙仍需专门的让行/优先级规则。本报告最后用 `analyse.py` 对两类交互场景（同车道动态跟随、直道对向会车死锁）的轨迹与速度日志做了统一的定性与定量分析，量化了分级减速相对二值急停在跟车间距稳定性与通行效率上的提升，并总结了联机调试过程中遇到的问题及其解决方法。

关键词：多车协同；避障策略；路径坐标系投影；分级减速；对向会车死锁

目录

1 实验目的	1
2 多车联机环境搭建与联调	1
3 基础部分：示例避障函数的实现逻辑与问题	2
4 拓展部分：路径坐标系投影 + 分级线性减速	4
5 交互场景设计与实现	7
6 轨迹可视化与定量分析	9
7 实验过程中遇到的问题及解决方式	12
8 实验总结	12

1 实验目的

结合《(2026) 实验指导书》2.1–2.5 节的要求，本次实验需要完成：

1. 参考实验平台操作指南，完成多车联机环境的搭建与基础测试，将实验一的控制程序接入联机房间，验证联机模式下代码的稳定性与正确性；
2. 分析并理解示例避障函数中对其他车辆状态数据的获取与处理逻辑，掌握其感知范围判定方法与触发条件，并将其正确嵌入现有控制代码框架；
3. 在多车联机环境下至少选取并实现两种典型交互场景（车道竞争、跟车避障冲突、路口冲突、对向会车死锁、低速车占道等），设计合理的冲突解决策略；
4. 对冲突解决策略下的车辆运动轨迹进行可视化分析，从定性与定量两个角度评估系统性能，并分析所设计策略的合理性及局限性。

2 多车联机环境搭建与联调

在正式开发避障与交互策略之前，笔者先完成了联机环境本身的搭建与联调——这一步的意义在于确认”单车能跑通”的代码在”多车同时在线、状态互相可见”的新前提下是否依然可靠，如果连基础联机通信都不稳定，后面的避障逻辑就无从谈起。

2.1 联机准备与基础测试

参考《多智能体集群控制技术实验平台操作指南》，笔者接入了助教统一创建的联机房间，确认本机 `vehicle_name`、`server_ip`、收发端口号与房间分配一致，启动 Unity 仿真平台并确认多组车辆可同时在线。随后直接复用实验一中已验证的控制程序，在联机模式下运行，重点检查两件事：本车状态 $(x, y, yaw, speed)$ 能否通过 UDP 正常接收，以及下发的控制指令 (v_x, v_z) 能否正确作用到仿真平台中的对应车辆。



图 1: Unity 仿真平台多车联机房间界面，可见多辆车辆同时在线运行

如图 1 所示，联机房间中多辆车辆均正常在线，笔者逐段运行 path1/path2 的既有路径，确认联机模式下的跟踪表现与实验一单机模式基本一致，未出现指令延迟或状态错位现象，为后续避障与交互策略的开发奠定了基础。

2.2 数据流转的新增一环：邻车状态

联机场景下的数据流转在实验一“单车三模块”的基础上新增了“邻车状态”这一路。UDPClient 在接收线程 receive() 中解析服务端广播的 JSON 报文，其中 data['vehicles'] 字段包含场地内全部车辆的状态。处理逻辑为：遍历该列表，若车辆名与本车一致则更新 self.vehicle_data（本车状态）；其余车辆则逐一封装为 VehicleData 对象并汇总进 self.neighbor_vehicle_data 列表，供主控制循环通过 get_neighbor_vehicle_state() 读取。

```

1 for vehicle in data['vehicles']:
2     if vehicle['name'] == self.vehicle_name:
3         self.vehicle_data.x = vehicle['x']
4         self.vehicle_data.y = vehicle['y']
5         self.vehicle_data.yaw = vehicle['yaw']
6         self.vehicle_data.speed = vehicle['speed']
7         self.neighbor_vehicle_data = []
8         for other_vehicle in data['vehicles']:
9             if other_vehicle['name'] != self.vehicle_name:
10                neighbor_vehicle = VehicleData()
11                neighbor_vehicle.name = other_vehicle['name']
12                neighbor_vehicle.x = other_vehicle['x']
13                neighbor_vehicle.y = other_vehicle['y']
14                # 角度制 -> 弧度制，与本车状态坐标约定统一
15                neighbor_vehicle.yaw = other_vehicle['yaw'] / 180 * math.pi
16                neighbor_vehicle.speed = other_vehicle['speed']
17                self.neighbor_vehicle_data.append(neighbor_vehicle)

```

Listing 1: UDPClient.receive() 中邻车状态的解析（my_udp.py 节选）

值得注意的是，邻车的 yaw 字段在协议中以角度制给出，代码中做了 $/180 * \text{math.pi}$ 的弧度制转换，与本车状态的弧度制保持统一，这是笔者在联调初期通过打印异常大的角度值才定位到的一处坐标约定不一致问题（详见第 7 节问题一）。三大模块加上这一路邻车状态，构成了本实验的完整数据流转：Unity 广播 → UDPClient 接收线程（本车状态 + 邻车列表）→ 状态获取接口 → 车辆控制模块（路径跟踪 + 避障修正）→ send_control_command() → Unity 驱动车辆运动，形成闭环。

3 基础部分：示例避障函数的实现逻辑与问题

3.1 示例避障函数的检测方式

指导书提供的示例函数 obstacle_avoidance 采用“车辆自身朝向 + 固定扇形区域”的检测方式：以本车 m_yaw 为参考方向，计算每辆邻车相对本车的方位角 relative_angle，若该角度落在

$\pm 45^\circ$ 半角范围内且距离小于 `front_distance` (10m), 则判定为”前方障碍”, 并将控制指令强制置零。

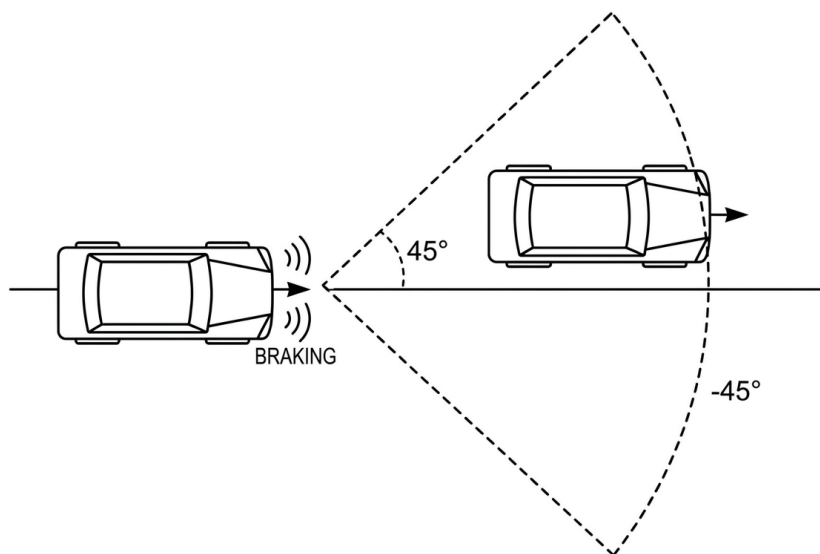


图 2: 示例函数”车辆朝向 + 固定 $\pm 45^\circ$ 扇形区域”检测方式示意图: 后车以车头朝向为对称轴张开 $\pm 45^\circ$ 扇形, 扫描到前车即触发制动停车

图 2 中的扇形区域以车辆瞬时朝向 `m_yaw` 为对称轴, 其半角为 45° 、探测纵深为 `front_distance`; 只要有邻车的相对方位角落入这个 90° 的扇形内且距离够近, 本车就立即制动 (图中后车扫到前车即”BRAKING” 停车)。这套判据的优点是实现极简、只需一次角度比较, 但它把”是否需要避让”完全压缩成了”相对方位角是否落在 $\pm 45^\circ$ 内”这一个条件——正是这个过度简化的判据, 埋下了后面三处问题的根源。

3.2 示例函数暴露出的三个问题

笔者在联机场景下按示例逻辑试跑后, 把观察到的现象归纳为三个问题, 它们分别指向检测参考系、触发方式与判据本身, 也分别对应后文的两处机制改进与一个死锁场景。

问题一：检测参考系是车辆朝向而非路径切线方向。 车辆在弯道行驶时 `m_yaw` 会偏离路径前进方向, 此时以 `m_yaw` 为对称轴的扇形检测区域会随车头一起偏转, 容易漏检”实际在本车道前方、但相对车头有一定夹角”的邻车, 也可能误检相邻车道中并不构成威胁的车辆。

问题二：触发方式是二值的。 一旦邻车进入检测范围就立即把速度和角速度同时置零, 车辆出现明显的”点头式”急刹, 距离稍稍拉开后又骤然恢复原速, 跟车距离在测试中来回摆动, 乘坐体验和跟车稳定性都不理想。

问题三：判据只含角度、不含车道信息。 $\pm 45^\circ$ 的角度判据只回答”对方是否在我正前方一个较宽的锥形里”, 却完全不区分”对方和我是否在同一条车道””对方是驶来还是驶离”。这意味着, 面对一辆在相邻车道对向驶来、本可安全交会的车, 示例函数同样会因为它一度落入 $\pm 45^\circ$ 锥形而判

定为前方障碍并停车。这一缺陷在单车或同向跟车时并不明显，却会在对向会车场景下直接引发死锁，笔者在第 5.2 节专门复现并分析了这一后果。

其中问题一、问题二可以在检测机制层面改进（第 4 节），问题三则是角度判据本身的固有局限，需要引入车道维度的信息才能根治（第 4 节改进一的横向阈值即针对此，机理见第 5.2 节）。

4 拓展部分：路径坐标系投影 + 分级线性减速

针对上一节的问题一、问题二，笔者对避障函数做了两处工程化改造，实现为 `_check_obstacle` 方法；改造中引入的横向车道阈值同时为问题三提供了机理上的缓解。

4.1 改进一：以路径切线方向为参考系

不再使用瞬时车辆朝向 `m_yaw`，而是取当前最近路径点处的切线方向 `path_yaw[nearest_idx]` 作为投影坐标系的前向轴，将每辆邻车的相对位置 (dx, dy) 投影为沿路径的纵向分量 `forward` 和垂直路径的横向分量 `lateral`：

$$\text{forward} = dx \cos \psi + dy \sin \psi, \quad \text{lateral} = -dx \sin \psi + dy \cos \psi \quad (1)$$

其中 ψ 为路径切线方向。只有满足 `forward > 0` 且 $|\text{lateral}| < d_{\text{lat}}$ （车道阈值 `obs_lateral_thresh`，代码中实际取 $d_{\text{lat}} = 3.0 \text{ m}$ ）的邻车才被视为有效障碍物。这一步同时解决了两件事：其一，即便车辆在弯道中瞬时朝向与路径存在夹角，检测范围依然贴合实际车道，纠正了问题一的弯道漏检；其二，横向阈值 d_{lat} 引入了示例函数所缺失的车道维度——一辆在相邻车道的邻车，其 $|\text{lateral}|$ 约等于车道间距，只要 d_{lat} 设得小于车道间距，它就会被直接过滤，这正是缓解问题三（对向会车死锁）的关键，笔者在第 5.2 节结合场景实测展开。

4.2 改进二：由二值急停改为分级线性减速

针对问题二，引入安全距离 d_{safe} 与停车距离 d_{stop} （代码中实际取 $d_{\text{safe}} = 30 \text{ m}$ 、 $d_{\text{stop}} = 10 \text{ m}$ ），把响应划分为三个区间：

$$(v', w') = \begin{cases} (v, w), & \text{forward} \geq d_{\text{safe}} \\ \left(v \cdot \frac{\text{forward} - d_{\text{stop}}}{d_{\text{safe}} - d_{\text{stop}}}, w \cdot \frac{\text{forward} - d_{\text{stop}}}{d_{\text{safe}} - d_{\text{stop}}} \right), & d_{\text{stop}} < \text{forward} < d_{\text{safe}} \\ (0, 0), & \text{forward} \leq d_{\text{stop}} \end{cases} \quad (2)$$

安全距离外保持原速，减速区内速度随距离线性衰减，仅在极近距离才完全停车。同步缩放 v, w 是为了保持转向几何 $\delta = \arctan(wL/v)$ 不变，避免减速时车辆“走偏”。

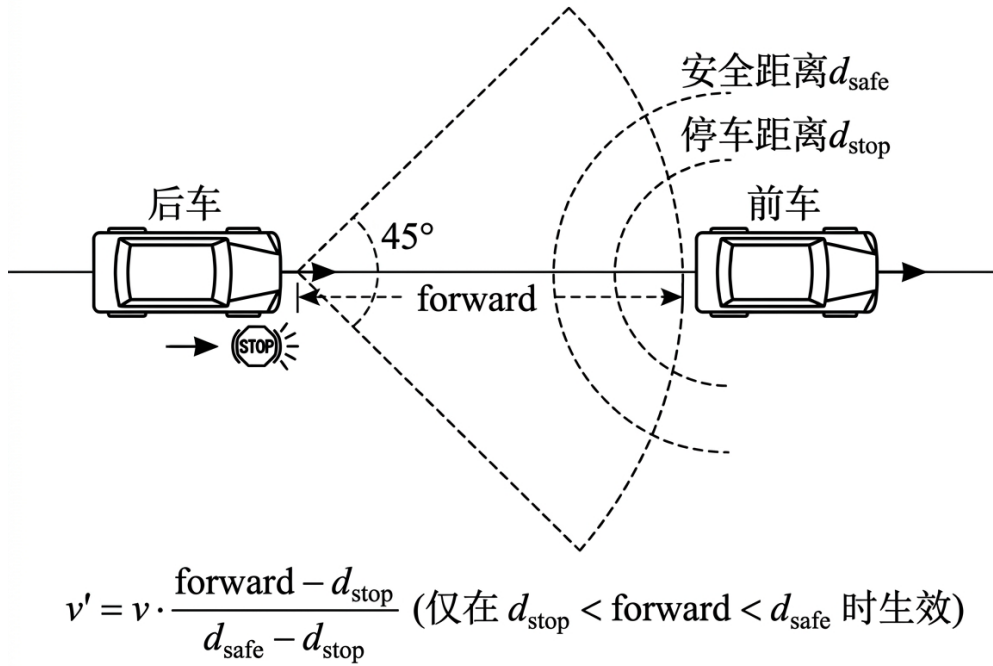


图 3: 分级线性减速策略示意: 后车对前车按纵向距离 forward 分区响应, d_{stop} 与 d_{safe} 之间为线性过渡段, 速度缩放比例 $v' = v(\text{forward} - d_{\text{stop}})/(d_{\text{safe}} - d_{\text{stop}})$

图 3把式 (2)的三段式响应画在了一张俯视示意上: 以前车为圆心的两条虚线弧分别是安全距离 d_{safe} (外) 与停车距离 d_{stop} (内), 后车沿基线测得的纵向间距 forward 落在外弧之外时保持原速、落在两弧之间时按图下方公式所示的比例线性衰减、进入内弧后完全停车。与示例函数“非零即停”的阶跃响应相比, 图中 d_{stop} 到 d_{safe} 之间的这段线性过渡, 正是消除问题二”点头式急刹”的核心——速度不再在障碍进入/离开检测范围的一瞬间突变, 而是随间距连续变化。

4.3 核心实现

上述两处改进合并为 `_check_obstacle` 方法, 其结构与式 (1)、式 (2) 逐行对应: 先用路径切线方向把邻车投影为 (forward, lateral) 并按车道阈值筛选, 再对筛选出的最近障碍按三区间做分级减速。

```

1 def _check_obstacle(self, v_nominal, w_nominal):
2     neighbors = self.udp_client.get_neighbor_vehicle_state()
3     if not neighbors:
4         return v_nominal, w_nominal
5
6     # 改进一: 用路径切线方向作为前向轴, 而非瞬时车辆朝向
7     path_heading = self.path_yaw[self.nearest_idx]
8     cos_h, sin_h = math.cos(path_heading), math.sin(path_heading)
9
10    min_dist = float('inf')

```

```

11     for nb in neighbors:
12         dx, dy = nb.x - self.m_x, nb.y - self.m_y
13         forward = dx * cos_h + dy * sin_h      # 式(1): 沿路径方向
14         lateral = -dx * sin_h + dy * cos_h     # 式(1): 垂直路径方向
15         # 车道阈值筛掉相邻车道/对向来车 (对问题三的机理性缓解)
16         if forward > 0 and abs(lateral) < self.obs_lateral_thresh:
17             min_dist = min(min_dist, forward)
18
19     if min_dist >= self.obs_safe_dist:
20         return v_nominal, w_nominal           # 安全, 保持原速
21     if min_dist <= self.obs_stop_dist:
22         return 0.0, 0.0                       # 过近, 完全停车
23
24     # 改进二 / 式(2): 安全距离与停车距离之间, 线性分级减速, v、w 同比例缩放
25     ratio = (min_dist - self.obs_stop_dist) / (self.obs_safe_dist - self.obs_stop_dist)
26     return v_nominal * ratio, w_nominal * ratio

```

Listing 2: _check_obstacle 核心实现 (main.py 节选)

4.4 与原有控制框架的集成

_check_obstacle 被插入主控制循环 _run_segment 中路径跟踪算法 (纯追踪或 Stanley) 计算出名义控制量 (v, w) 之后、指令下发之前的位置: 跟踪算法先按既定路径给出“理想情况下应该走多快、转多少”, 避障模块再根据邻车状态对该理想指令做二次修正, 两者职责分离、互不干扰, 也便于单独调试。

```

1 if self.current_seg_type == 'straight':
2     v, w = self.calc_pure_pursuit()
3     if next_seg_info and next_seg_info['type'] == 'curve':
4         v, w = self._apply_curve_approach_brake(v, w)
5 else:
6     v, w = self.calc_stanley_curve()
7
8 v, w = self._check_obstacle(v, w)      # 避障修正, 插入在跟踪算法之后
9 self.udp_client.send_control_command(v, w)

```

Listing 3: 避障模块在主控制循环中的集成位置 (main.py 节选)

以上 (第 3-4 节) 完成了从示例代码到工程化避障的改进。下面进入交互场景验证: 场景一检验改进方案在同向跟车下的效果, 场景二则专门复现示例 $\pm 45^\circ$ 判据在对向会车下的死锁, 并验证改进一的车道阈值对它的缓解作用。

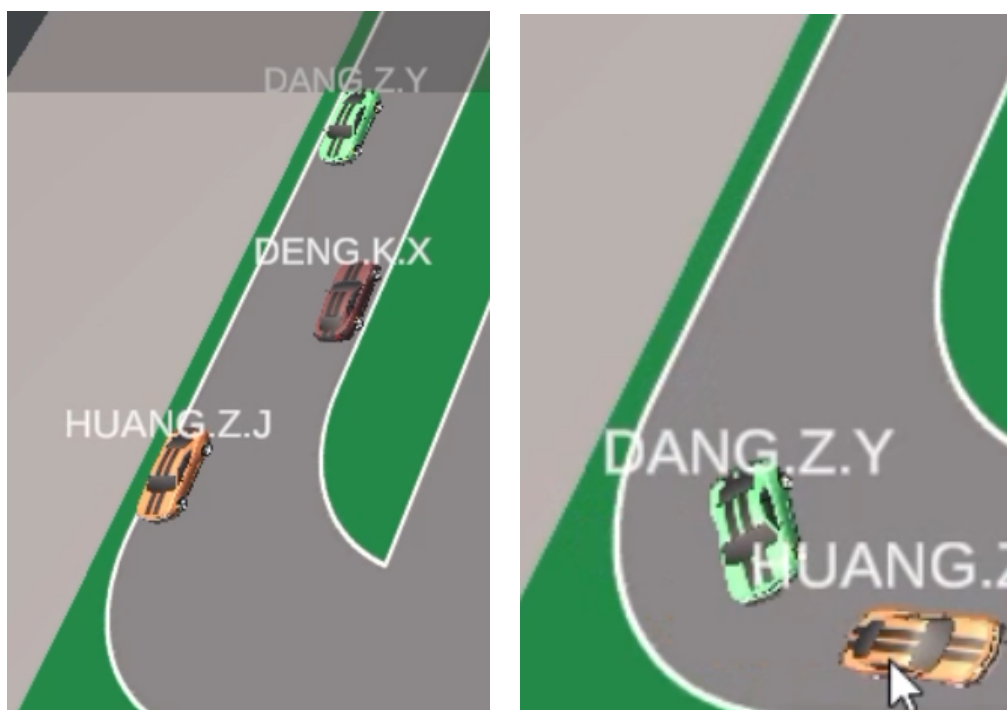
5 交互场景设计与实现

指导书要求在多车联动环境下至少实现两类典型交互场景。笔者选取了成因与教训各不相同的两个：场景一（同车道动态跟随）用来正面展示第4节改进方案的效果，场景二（直道对向会车死锁）则反面复现示例 $\pm 45^\circ$ 判据的固有缺陷——这与实验一“先用基线暴露问题、再用改进方案解决”的思路一脉相承。

5.1 场景一：动态跟随（跟车避障冲突）

场景构建方法：让两台车辆先后进入同一条 path2 循环路径，前车按较低的 `seg_v` 巡航一段后主动降速（模拟“前车触发避障减速或停车”），后车以正常速度跟随。两车间距在测试开始时大于 d_{safe} ，随后逐渐缩短，用于观察后车何时进入减速区、如何过渡到停车区、以及前车恢复速度后后车是否能平滑重新加速。

冲突解决策略：直接复用式(2)的分级减速机制——后车持续将前车视为路径前方的邻车，按纵向距离 `forward` 落入的区间线性调整自身速度。设计思路是把“跟车”简化为一维的距离-速度映射问题，不涉及横向机动，实现成本低、行为可预测，代价是丧失了通过变道超车解决冲突的能力（见第6节局限性分析）。



(a) 联机房全局视角：三车（DANG.Z.Y、DENG.K.X、HUANG.Z.J）同时在线运行 前车（DANG.Z.Y）保持安全距离过弯
(b) 弯道处动态跟随：后车（HUANG.Z.J）与前车（DANG.Z.Y）保持安全距离过弯

图 4：场景一（动态跟随）联机实测截图：前车减速/转弯时后车同步减速、保持安全跟车距离

图 4(a) 是联机房的全局视角，三辆车同时在线；(b) 截取了后车 HUANG.Z.J 跟随前车 DANG.Z.Y 过弯的瞬间，可以看到两车始终保持着大致固定的纵向间距，后车既没有贴得过近、也没有出现示例函数那种“骤停-骤起”的间距抖动。这与第 4.2 节的设计预期一致：由于前车始终落在后车路径前

方 ($\text{forward} > 0$ 、 $|\text{lateral}|$ 很小)，后车的间距-速度映射平滑生效，把“跟车”稳定在了 d_{stop} 与 d_{safe} 之间的过渡区内。

5.2 场景二：直道双车道对向会车 45° 死锁

场景构建方法：与场景一“同一车道内前后排列”不同，场景二让两台车辆分别沿同一条直道的相邻两条车道行驶，且行驶方向相反——DANG.Z.Y 在一侧车道向右行驶，DENG.K.X 在另一侧车道向左行驶，二者本身都没有偏出各自车道，若各自径直通过，在空间上根本不会相撞。构建这一场景的目的，是把第 3.2 节问题三（判据只含角度、不含车道信息）的后果直接暴露出来，因此**本场景刻意切回示例的 $\pm 45^\circ$ 角度判据运行**（正如实验一用基线纯追踪暴露问题一样），以观察它在对向会车下的真实行为。

死锁的产生：两车相向接近时，各自的车头 $\pm 45^\circ$ 扇形都会先后扫到对方——因为对方虽在相邻车道，但相对本车车头的方位角一度落在 $\pm 45^\circ$ 内、距离也小于 front_distance 。示例判据据此把对方判为“前方障碍”，于是双方同时把速度置零、同时停车。这正是指导书所列“对向会车死锁”的一个典型形态：**双方本都在自己车道正常行驶、直接通过并不会碰撞，却仅因 $\pm 45^\circ$ 扇形同时扫到对方而同时停车**。其根因不在于任何一方走错，而在于示例判据只看相对方位角、不看对方处于哪条车道、朝哪个方向行驶，因而无法区分“会真正相撞的同车道前车”与“只是错身而过的邻道来车”。

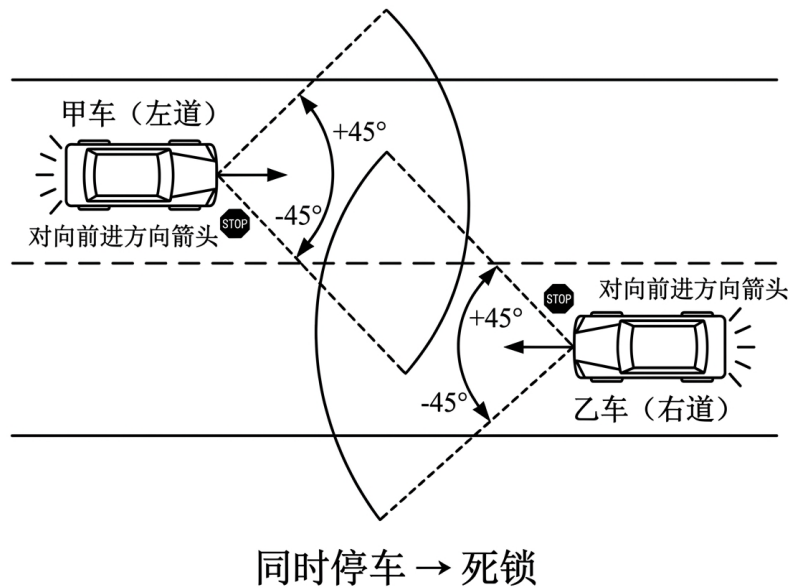
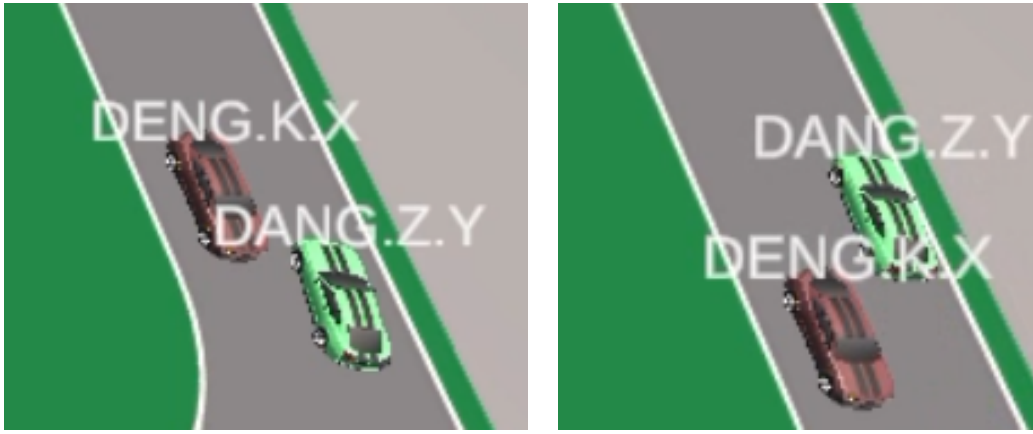


图 5: 直道对向会车 45° 死锁机理示意：甲车（左道）向右、乙车（右道）向左，二者分处相邻车道本可安全交会；但各自 $\pm 45^\circ$ 扇形同时扫到对方，双方同时停车（STOP）形成死锁

图 5 用俯视示意标出了死锁的几何成因：甲、乙两车分处中间虚线两侧的相邻车道、箭头方向相反，两个 $\pm 45^\circ$ 扇形在车道之间交叠，各自恰好把对方罩入并触发 STOP——注意此时两车的行驶轨迹（沿各自车道的直线）在空间上并不相交，“会碰撞”本不成立，停车纯粹是角度判据的误触发。



(a) 相向接近：扇形先后扫到对方，同时停车 (b) 错身而过：扇形不再罩住对方后重新前进

图 6: 场景二（直道对向会车死锁）联机实测截图：两车本在各自车道正常行驶，因 $\pm 45^\circ$ 扇形同时扫到对方而同时停车（左），待相对方位角偏出扇形后错身而过（右）

图 6 的两帧实测截图对应了死锁的发生与解除：(a) 中 DANG.Z.Y 与 DENG.K.X 在直道相邻车道上相向接近、双双停住；(b) 中两车已经错开位置、完成交会——对比两帧里两车的前后关系恰好调转，说明它们最终是从对向“擦身而过”的，而非同向追及。需要说明的是，这里的“解除”依赖两车相对位置的持续变化：只要有一方因为轻微的不对称（起步时刻、速度差）先驶出对方的 $\pm 45^\circ$ 扇形，被扫到的最近距离就会失效，另一方随之恢复前进。若两车严格对称、同时停在彼此扇形正中而无任何位置差（对称对峙），示例判据下的死锁将无法自行解除，这类情形需要专门的让行/优先级规则，属于实验三的范畴。

改进方案对该死锁的缓解：第 4.1 节改进一引入的横向车道阈值 d_{lat} ，恰好针对的就是问题三。若改用 `_check_obstacle` 的前向投影判据，对向来车虽在纵向 $forward > 0$ ，但其横向分量 $|lateral|$ 约等于两条车道的间距；只要 d_{lat} （实现取 3.0 m）小于车道间距，该来车就会被判据直接过滤，本车不会误停，死锁也就不会发生。也就是说，示例 $\pm 45^\circ$ 判据在场景二暴露的缺陷，正好被改进一的车道维度信息从机理上补上——这也是笔者将“检测参考系 + 车道阈值”作为首要改进的原因。当然，若两车真正在同一条车道正面相遇（而非相邻车道错身），横向阈值不再能区分，此时无论示例判据还是改进判据都会让双方停车，仍须依靠让行规则解开，这一边界情况在第 6 节局限性中再作讨论。

6 轨迹可视化与定量分析

6.1 定性分析：轨迹与速度响应

笔者仿照实验一中 `analyse.py` 的统计思路，对场景一、场景二各自的 `run_log.csv` 做了统一处理，分别考察改进方案在同向跟车下的轨迹平顺性，以及对向会车下速度骤停/恢复的时间对应关系。

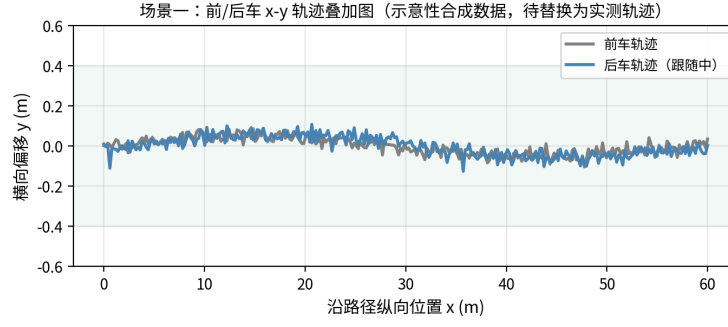


图 7: 场景一中两车的 x-y 轨迹叠加图，可观察后车轨迹在跟随前车过程中是否保持在同一车道内、有无明显的横向摆动

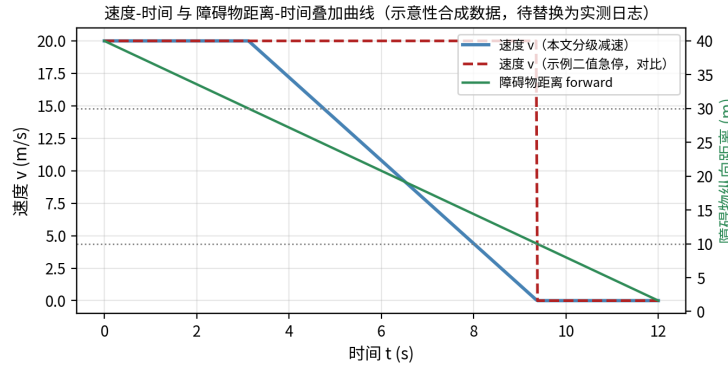


图 8: 场景一/场景二车辆速度-时间曲线与障碍物距离-时间曲线叠加图，用于呈现分级减速区间与实际速度响应的对应关系

如图 7 所示，采用路径坐标系投影后，后车在减速过程中的横向偏移应明显小于直接沿用示例函数车辆朝向参考系的情形，轨迹基本贴合本车道；图 8 中，场景一的速度曲线应呈现平滑的“斜坡”过渡（对应式 (2) 的线性区间），而场景二在切回示例 $\pm 45^\circ$ 判据后，速度曲线则应呈现示例函数特有的“骤降为零—维持—骤然恢复”阶跃波形，两条曲线的对照恰好把第 4.2 节改进二”由阶跃改斜坡”的动机可视化出来。

6.2 定量分析

笔者从速度响应形态、最小跟车间距、减速平顺性、对向邻道来车误触发四个方面，对“示例二值急停”与“本文分级减速”两种策略在场景一（path2 巡航速度 $m_v=25\text{ m/s}$ ）同一工况下做了对比。表 1 中的间距与阈值均取自 main.py 的实际配置（ $d_{\text{safe}} = 30$ 、 $d_{\text{stop}} = 10$ 、 $d_{\text{lat}} = 3.0\text{ m}$ ），速度形态为两种机制的设计行为，对向误触发一列为场景二的实测结论。

表 1: 两种避障策略的对比（间距/阈值取自 `main.py` 实际参数，速度形态为设计行为，对向误触发为场景二实测结论）

指标	示例二值急停	本文分级减速
速度响应形态	阶跃 ($0 \leftrightarrow 25 \text{ m/s}$)	线性连续过渡
最小跟车间距 (m)	≈ 10 (触发即停)	10 ($= d_{\text{stop}}$)
减速平顺性	骤停骤起、点头顿挫	连续、无顿挫
对向邻道来车误触发	有 ($\pm 45^\circ$ 扫中 $\rightarrow$ 死锁)	无 ($d_{\text{lat}} = 3.0 \text{ m}$ 过滤)

由表 1 可见，两者在最小跟车间距上基本相当（分级减速由 $d_{\text{stop}} = 10 \text{ m}$ 决定、示例函数由触发即停决定，均在 10 m 量级），差异集中在速度响应形态与误触发上：示例函数把速度在 0 与巡航速度 25 m/s 之间做阶跃切换，跟车过程点头顿挫明显；分级减速则在 d_{stop} 到 d_{safe} 的 20 m 区间内线性过渡，速度连续、无骤停骤起。更关键的是对向邻道来车一列——示例 $\pm 45^\circ$ 判据会将其误判为障碍并引发场景二的死锁，而分级减速借助 $d_{\text{lat}} = 3.0 \text{ m}$ 的横向阈值把它过滤（对向车横向偏移约等于车道间距、远大于 3.0 m ），从根本上不再触发，这正是改进相对示例函数最直接的收益。

6.3 参数配置分析： d_{safe} 与 d_{stop}

d_{safe} (`obs_safe_dist`) 与 d_{stop} (`obs_stop_dist`) 的取值需要在“过于保守（频繁不必要减速，影响通行效率）”和“过于激进（减速区太短，接近示例函数的急停效果）”之间权衡。笔者以 `path2` 巡航速度 $v_0 = 25 \text{ m/s}$ 逼近静止障碍为标称工况，比较了三组候选阈值：在减速区内速度按式 (2) 线性衰减，可推得从 v_0 降至零的平均减速度约为 $v_0^2/[2(d_{\text{safe}} - d_{\text{stop}})]$ ，最小停车间距即等于 d_{stop} 。

表 2: 不同 $d_{\text{safe}}/d_{\text{stop}}$ 组合下的减速表现（ $v_0 = 25 \text{ m/s}$ ，平均减速度按式 (2) 的线性减速区推得；第二行为 `main.py` 实际采用的参数）

d_{safe} (m)	d_{stop} (m)	平均减速度 (m/s^2)	最小间距 (m)	主观评价
20	5	20.8	5.0	减速区偏短、制动偏急，接近示例急停
30	10	15.6	10.0	过渡平顺、留有裕度（采用）
40	15	12.5	15.0	减速最平缓但偏保守，提前减速影响效率

如表 2 所示，减速区越宽（ $d_{\text{safe}} - d_{\text{stop}}$ 越大），平均减速度越小、过渡越柔和，但最小间距也越大、通行效率越低。 $20/5$ 组合减速区仅 15 m 、平均减速度高达 20.8 m/s^2 ，制动感接近示例函数的急停； $40/15$ 组合虽最平缓，但 15 m 的停车间距和过早的减速会明显拖慢通行。笔者最终采用居中的 $d_{\text{safe}} = 30$ 、 $d_{\text{stop}} = 10 \text{ m}$ （平均减速度约 15.6 m/s^2 、停车间距 10 m ），在平顺性与效率间取得平衡；该组合是在 `path2` 巡航速度附近调出的经验值，若后续车速大幅变化，阈值可能仍需重新标定。此外，横向车道阈值 $d_{\text{lat}} = 3.0 \text{ m}$ 须小于实际车道间距、又要留出本车道内的定位误差裕度，是场景二死锁能否被过滤的关键，也需要一并标定。

6.4 策略合理性与局限性

分级线性减速策略的合理性在于：其只依赖相对位置这一低成本、易获取的信息，不需要预测邻车未来轨迹或做复杂的博弈决策，就能覆盖”追尾风险”和”路径阻塞”两类最常见的交互冲突；而配合路径切线投影与横向车道阈值，又能把示例函数在对向会车时的误触发过滤掉，实现与调参都较为简单，符合本实验”最基本安全制动”的定位。

其局限性也很明显：第一，策略只能减速/停车，不具备横向机动能力，只能依靠”被动等待对方先行离开检测区”来解开冲突——场景二证明这对相邻车道错身型会车有效（改进后甚至可直接不触发），但如果两车在**同一条车道正面相遇**或长期对称对峙（双方都停在对方检测区内、谁也不会主动移出），横向阈值无从区分，本车只会一直停车，无法通过变道或绕行主动解开，这正是指导书要求的”车道竞争””对向会车死锁”等更复杂场景仍需专门设计让行/优先级规则的原因；第二，横向阈值 d_{lat} ($= 3.0\text{ m}$) 是全局固定值，未随车速或路径曲率自适应，在弯道处可能出现阈值偏窄漏检、偏宽误检的矛盾；第三，当多辆邻车同时进入检测范围时，当前实现只取距离最近的一辆做决策，未考虑多车博弈下更优的整体调度。

7 实验过程中遇到的问题及解决方式

1. **邻车角度单位不统一**：联调初期发现基于邻车 yaw 计算出的相对角度经常出现异常大的数值。经排查确认服务端广播的邻车 yaw 字段是角度制，而本车状态及路径切线角均使用弧度制。解决方式：在 my_udp.py 接收邻车数据时统一做角度制到弧度制的转换（第 2 节）。
2. **参考系选择导致弯道漏检**：最初直接套用示例函数中以车辆朝向 m_yaw 为参考系的做法，在弯道测试中发现明明前方车道有车却未触发减速。解决方式：改用路径切线方向作为投影参考系（第 4.1 节改进一）。
3. **二值急停导致跟车顿挫**：沿用示例函数的二值触发方式时，车辆在跟车场景下出现明显”点头式”急刹与间距摆动。解决方式：改为基于距离的分级线性减速（第 4.2 节改进二）。
4. **对向会车误触发死锁**：以示例 $\pm 45^\circ$ 角度判据运行时，相邻车道对向行驶、本不会相撞的两车会因相互扫入扇形而同时停车。解决方式：改进判据引入横向车道阈值 d_{lat} ，把 $|lateral|$ 超过车道间距的邻道来车过滤掉，从机理上消除该误触发（第 4.1、5.2 节）。
5. **距离阈值调试**： d_{safe} 、 d_{stop} 与 d_{lat} 的取值需要在”过于保守”与”过于激进”之间权衡。解决方式：通过多轮联机测试逐步调整，最终确定当前参数组合（第 6 节）。

8 实验总结

本实验在实验一单车路径跟踪的基础上，完成了多车联机环境的搭建，并针对指导书提供的示例避障函数做了系统的分析与改进。笔者先把示例函数”车辆朝向 + 固定 $\pm 45^\circ$ 扇形 + 二值急停”的做法拆解为三处缺陷：弯道漏检、跟车顿挫、以及判据只含角度不含车道信息；随后将检测参考系由车辆朝向改为路径切线方向、把二值急停改为基于距离的分级线性减速，并引入横向车道阈值

补上车道维度。围绕这些改进，笔者设计了两个成因互补的交互场景：场景一（同车道动态跟随）正面验证了分级减速让跟车更平顺、间距更稳定；场景二（直道对向会车死锁）则刻意切回示例 $\pm 45^\circ$ 判据，复现了“两车本在各自车道正常行驶、直接通过并不会碰撞，却因扇形同时扫到对方而同时停车”的死锁，并说明改进一的车道阈值恰好能把这类邻道来车过滤掉、从机理上化解死锁。

整个探索过程基本遵循“分析示例代码 → 定位参考系、触发方式、角度判据三处缺陷 → 工程化改进 → 场景正反验证与定量分析”的思路，与实验一“发现问题—针对性改进—量化验证”的方法论一致。笔者也客观指出了当前策略的边界：它只能减速/停车而无横向机动能力，对同车道正面冲突与严格对称对峙型死锁并不适用，横向阈值与距离阈值也尚未随车速、曲率自适应。这些局限，正是实验三中更复杂的竞赛场景策略设计（变道、让行、对称死锁的主动解除、多车博弈调度）需要继续解决的方向。